

Architectural Style Justification

Lab 3: Component Modelling and Architectural Pattern Selection

Scenario: OpenAPI Mock Server Generator (Problem Statement 41)

Name: Sidhant Holla

SRN: PES1UG24AM466

Architecture Selection

I chose Microservices Architecture for the OpenAPI Mock Server Generator System.

Architectural Choice

The system is decomposed into independently deployable services - an API Gateway, a Specification Management Service, a Mock Runtime Service, a Latency and Fault Configuration Service and a Logging and Metrics Service, together with an AES-256 encrypted Specification Store - communicating over REST APIs and an asynchronous queue, rather than as a single layered application or a centralised client-server model.

Two Reasons

- 1. Management traffic and mock traffic have completely different load profiles.** Uploading and validating a specification happens a handful of times a day per developer, but the mock endpoints that specification generates have to sustain 1000 requests per second inside CI runs. As separate services the Mock Runtime can be replicated across several instances to meet that throughput while Specification Management stays at one. In a layered monolith the two would scale together, so reaching mock throughput would mean paying for validation capacity that is never used, and a client-server split does not help because the bottleneck is one part of the server rather than the server as a whole.
- 2. The system deliberately misbehaves, so fault isolation is a requirement rather than a nicety.** The purpose of the tool is to inject faults and delays on purpose: an endpoint can be configured to stall for up to ten seconds or to return 503 for a set percentage of requests. If that ran in the same process as specification management and logging, a mock held open by design would consume threads the management console and the audit log need. Isolating the Mock Runtime as its own deployable service means a mock behaving badly, whether intentionally or not, cannot bring down the rest of the tool.

Security Advantage

Uploaded specifications describe internal APIs that have not yet been released, so a leak would expose the real system. The Mock Runtime Service has the largest exposed surface because it answers high volumes of traffic, yet it holds no credentials for the Specification Store and has no read path to it; it receives only the compiled route table it needs to answer requests. Authentication and authorisation for every management operation are enforced at a single boundary, the API Gateway, and the specifications themselves stay behind Specification Management in an encrypted store. Compromising the most exposed component therefore does not expose the specifications.

Performance Benefit

Simulated latency is only meaningful if the engine's own overhead is predictable, and NFR-001 targets accuracy within ten milliseconds. A dedicated Mock Runtime Service doing nothing but matching routes and returning generated responses has no specification parsing, no database writes and no user interface work competing for CPU in the same process, so the delay a tester measures is the delay they configured rather than that plus whatever else the process happened to be doing. Log records are handed to the Logging and Metrics Service over an asynchronous queue, so writing a row never adds to the time on the response path.

Component Diagram - OpenAPI Mock Server Generator

Microservices Architecture (Problem Statement 41)

